Lista 3 - Análise de Algoritmos

Aluno: Anderson Vicente de Freitas
Professor: Alexandre Freire

1- O método da **Árvore de Recursão** nos dá a forma visual de como as recorrências ocorrem em um algoritmo, e usa a soma dos custos de trabalho de cada nível para definir sua complexidade. Tem como vantagem a visualização da complexidade de forma mais clara por conta da sua visualização ser de forma gráfica, porém a depender do caso que será trabalhado, esse método pode se tornar custoso e exaustivo. O **Método Master** é como se fosse um "guia rápido" de como o algoritmo vai se comportar de acordo com sua função, onde a depender da sua estrutura e valores de a e b na função **T(n) = aT(n/2) + f(n)**, pode cair em 3 casos diferentes. Tem vantagem de sua resolução ser mais direta em comparação com a árvore, pois tem muita chance de cair nos três casos fundamentais, porém, não irá funcionar para todas as recorrências possíveis, visto que é utilizado mais em casos especiais do que como uma regra geral.

2- a)Algoritmo: Merge Sort (Ordenação por Intercalação). É um algoritmo de ordenação baseado em divisão e conquista, divide a lista desordenada de forma recursiva em duas metades até obter sublistas de um único elemento,e em seguida intercala de forma ordenada parra reconstruir a lista completa

b)Recorrência: Forma **T(n) = 2T(n/2) + θ(n)** com **a**(numero de chamadas recursivas) = 2, **b**(cada subproblema) = 2 e **f(n)**(trabalho da função Merge) = θ(n)

c)Aplicando o Teorema Master, descobrimos o logaritmo de 2 na base 2, que é igual a 1, logo n^log 2 2 = n¹ = **n.** Quando compararmos f(n) com n log b a, como f(n) = θ(n) e log b a = n, ambas vão ter a **mesma** ordem de crescimento caindo no caso 2 do teorema, e sua complexidade resultante fica T(n) = θ(n log b a * log n), ou seja, **T(n) = θ(n log n)**

d)Fonte: https://faculty.cc.gatech.edu/~ladha/algo/L2.pdf

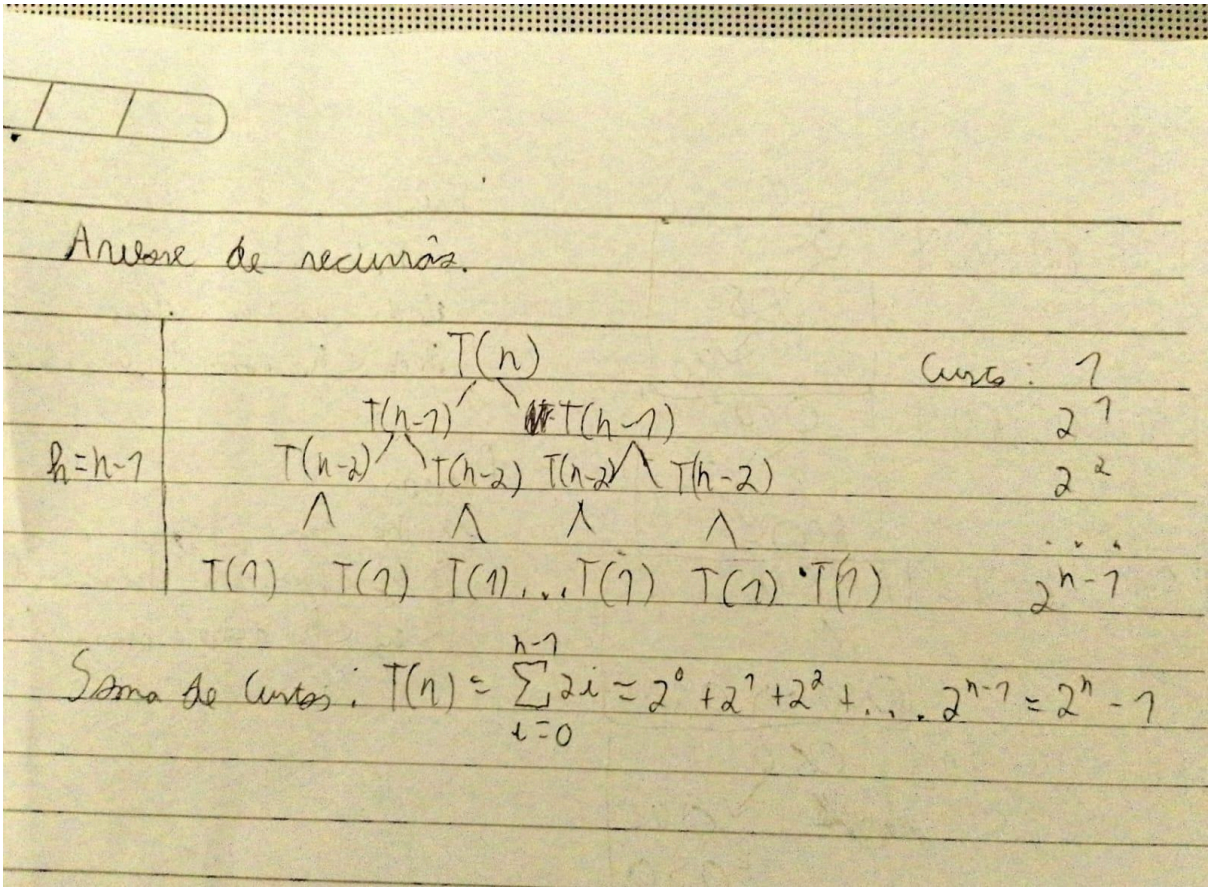
3- a) Algoritmo: Torres de Hanói (`MoveTower`). Tem como objetivo transferir uma pilha de n discos furados de um pino de origem para outro de origem com a ajuda de um pino auxiliar. **Recorrência: T(n) = 2T(n-1) + 1**

b) Estrutura da árvore: Nível 0(raiz): custo 1**, Nível 1:** 2 subproblemas com tamanho n - 1, cada um com custo 1, e custo total por nível: 2¹ = 2**, Nível i:** possui 2^i nós, cada um ganhando custo unitário 1 e custo total do nível sendo 2^i. O

tamanho da entrada diminui em 1 a cada passo(n-1, n-2…), o caso base T(1) é atingido após n - 1 reduções, logo terá uma altura exata de n -1 níveis.

c)Soma dos custos:

Complexidade assintótica: T(n) = θ(2^n)(exponencial). como o número de chamadas recursivas dobra a cada nível sem que o tamanho do problema seja dividio(apenas reduzido em 1), a quantidade total de operações cresce de forma exponencial.



d)Fonte: http://www.decom.ufop.br/haroldo/paa/paa-recurs.pdf

4-Algoritmo da questão 3: Torres de Hanói, com recorrência T(n) = 2T(n -1) + 1

a)Hipotese indutiva, provar por indução que **T(n) = O(2^n)**, ou seja, que existe um c>0 tal que **T(n) <= c * 2^n - 1**, para todo n>=n0, como hipótese assumimos que a desigualdade é válida para todos os valores menores que n, especificamente para k = n - 1, ou seja, T(n-1) <= c* (2^n-1) - 1

b)Passo Indutivo, em relação a recorrência original do algoritmo **2T(n -1) + 1,** substituimos a hipótese da indução no lugar de T(n - 1), logo T(n) <= 2*(c*(2^n-1) - 1) + 1 -> **T(n) <= * 2^n - 1**, com a expressão obtida sendo exatamente o limite pretendido.

c)Caso Base, para n = 1, T(1) = 1, já que o algoritmo move um disco por vez a cada passo. Logo, T(1) <= c * 2^1 - 1 -> 1 <= 2c -1 -> 2 <= 2c -> **c>=1**, então é satisfeito para qualquer constante c>=1.

d)Constantes c e n0 são respectivamente 1 e 1 para indução matemática ser válida e provar que T(n) = θ(2^n)